

TutorAid

Gaby Norris 241143

Open Window, Creative Technologies

DV 200

Tsungai Katsuro

24 July 2025

Table of Contents

B) Client Conceptualisation & Problem Statement.....	3
Overview of the Project & Business Domain:.....	3
Core Problem/Need:.....	3
Why the Software Solution Is Necessary:.....	3
Constraints or Limitations:.....	3
Name & Basic Logo Concept:.....	3
C) System Architecture.....	5
High-Level System Design:.....	5
Technologies & Frameworks:.....	5
Justification for Stack:.....	5
D) Feature Requirements & Scope.....	5
Scope of the System:.....	5
SMART Objectives:.....	6
List of Major Features:.....	6
Feature Prioritisation:.....	6
User Roles:.....	7
E) Data Planning.....	7
Entity-Relationship Diagram (ERD):.....	7
Key tables and their relationships.....	7
Data types and constraints for key fields.....	8
F) Wireframes & UI/UX Considerations.....	10
Moodboard.....	10
Basic wireframes of key screens.....	11
Justification for design decisions.....	12
Accessibility concerns.....	12
G) Project Timeline & Workflow.....	12
High-level Gantt chart.....	12
Milestones and Deliverables.....	13
Estimated Timelines.....	14
Project Management.....	14
H) Risks, Challenges & Conclusion.....	14
Potential Risks.....	14
Mitigation Strategies.....	15
Final Thoughts.....	15

B) Client Conceptualisation & Problem Statement

Overview of the Project & Business Domain:

TutorAid is a tutor-centred web application created to help tutors organise their lessons more easily. The app lets tutors list subjects, set availability, accept student bookings, track student progress, and receive feedback.

Core Problem/Need:

Many tutors struggle with administrative problems that can make it difficult to teach students. These challenges include organising students across different apps and calendars, handling cancellations and no-shows, keeping track of each student's progress and goals, and offering multiple subjects for learning but lacking a structured system.

Why the Software Solution Is Necessary:

The app simplifies workflows, which allows the tutor to focus more on teaching, streamlines student management and lesson bookings, helps the tutors keep up professionalism by tracking progress and lesson histories, it motivates structured learning through goal setting for students, and it reduces the chance of no-shows with automatic reminders and strict booking policies.

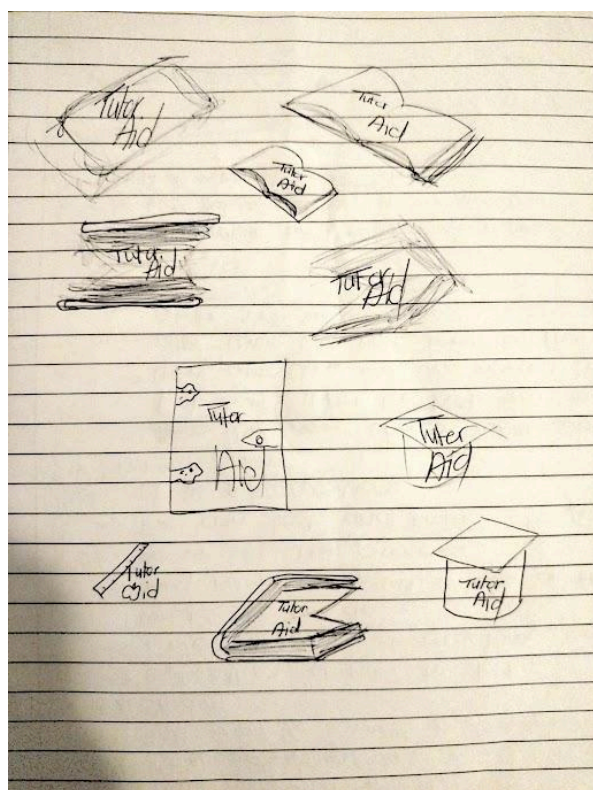
Constraints or Limitations:

Due to the system focusing on the tutor, students are only limited to booking a lesson and giving feedback. Tutors will have to manually organise their availability. There are no payment features as it is a part of the nice-to-haves.

Name & Basic Logo Concept:

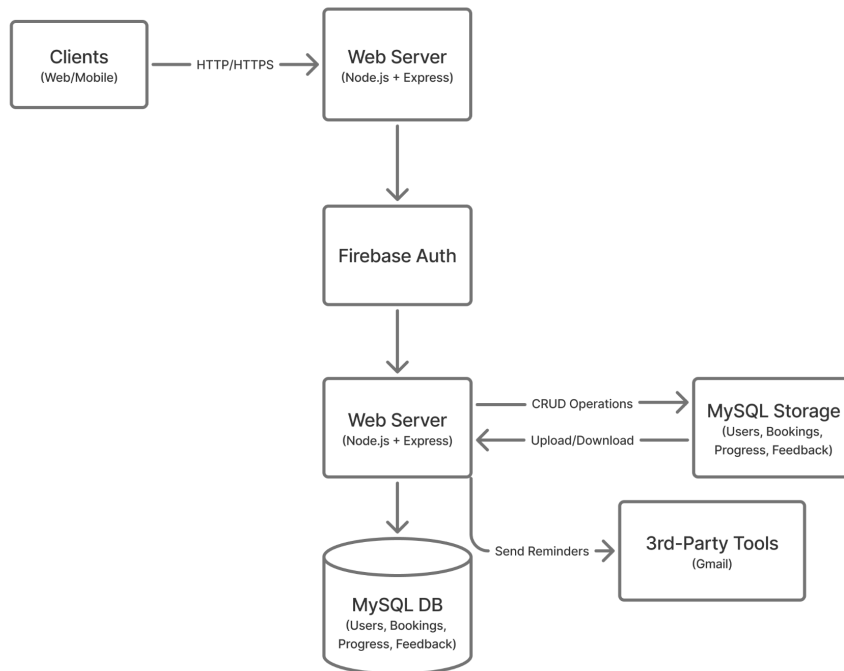
Name: TutorAid

Basic Logo Concept:



C) System Architecture

High-Level System Design:



Technologies & Frameworks:

- React, CSS, Node.js, Express, Firebase

Justification for Stack:

This stack enables quick development with real-time user interface feedback and is scalable and well-documented. The development process will run more smoothly if well-known frameworks like React and Node.js are used, but the developer will still have to manage state management, routing, and API connections.

D) Feature Requirements & Scope

Scope of the System:

Included:

- Tutor registration and profile management
- Subject management per tutor
- Availability setting and booking management
- Progress note tracking per student
- Lesson history and status updates
- Feedback collection from students

Excluded (MVP):

- In-app chat system (for lessons on Platforms like Google Meets, Teams or Discord)
- Group lesson handling
- Payment handling

SMART Objectives:

- Launch MVP by [Set Date]
- Allow students to book tutors in less than 3 minutes
- Enable tutors to log progress notes per student after each session
- Reduce scheduling conflicts by 90% through availability management

List of Major Features:

- Authentication - Signup/Login for users.
- Tutor Profiles - Displaying bios, subjects they teach and availability calendar.
- Booking System - Students can book lessons depending on tutor availability.
- Progress Notes - Tutors keep notes of the student's progress, privately for later evaluation.
- Lesson History - Keeps track of past and upcoming lessons.
- Feedback System - Students can leave feedback for tutors after the lessons.

Feature Prioritisation:

Must-Have (MVP): Authentication, Booking System and Progress Tracker

Nice-to-Have: Feedback and Calendar View

Future: In-app Chat System, Group Lessons and Payment Integration.

User Roles:

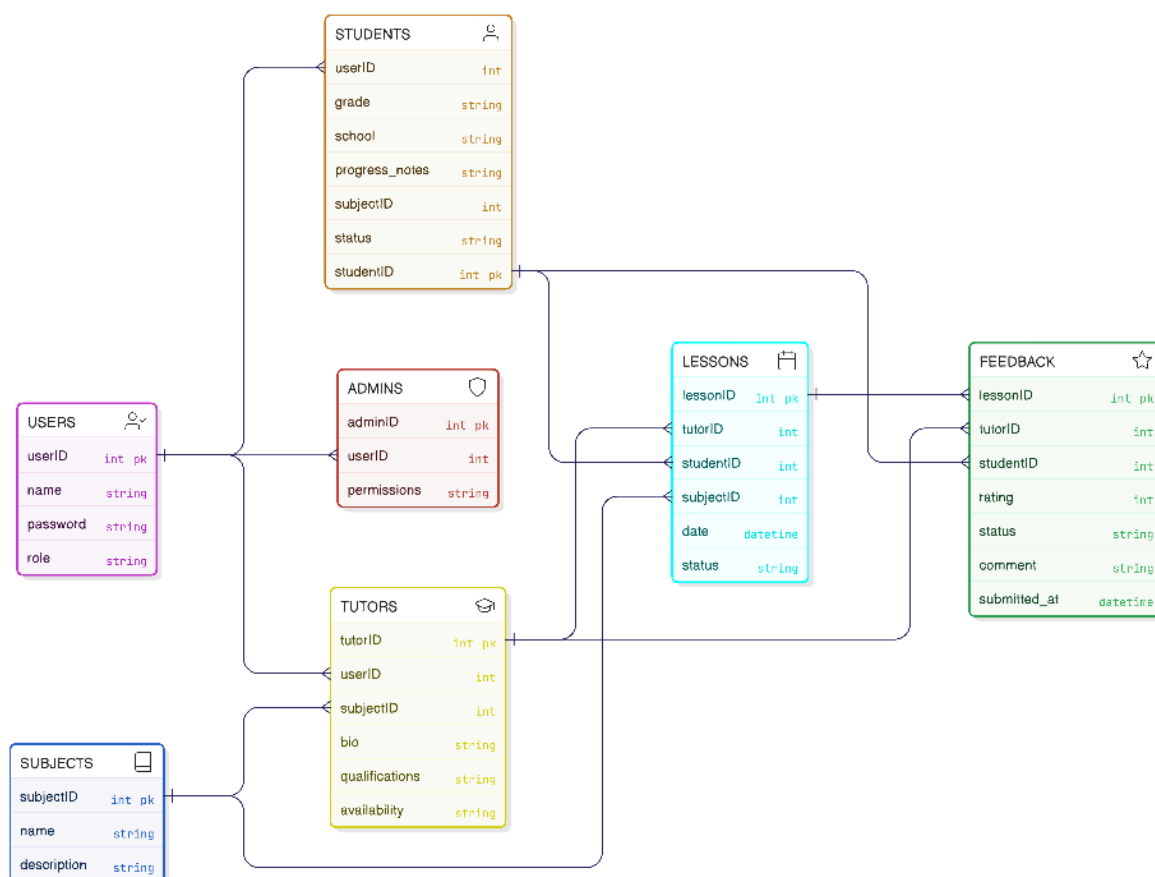
Admin: Does user verifications, handles reports from users, tracks the app's usage (monitoring for bugs) and ensures data integrity.

Tutor: Sets availability and subjects, manages booking and progress notes.

Student: Browses tutors, books lessons, attends lessons (external) and leaves feedback.

E) Data Planning

Entity-Relationship Diagram (ERD):



Key tables and their relationships

- Tutors (one-to-many) Lessons
- Students (one-to-many) Lessons
- Students (one-to-many) Progress Notes

- Lessons (one-to-one) Progress Notes
- Lessons (one-to-one) Feedback

Data types and constraints for key fields

USERS

Field	Data Type	Constraints
userID	Integer	Primary Key, required
name	String	Required
email	String	Unique, required
role	String	Required
password	String (hashed)	Required

ADMIN

Field	Data Type	Constraints
adminID	Integer	Primary Key, required
userID	Foreign Key Users(userID)	Required
permissions	Integer	Optional

TUTORS

Field	Data Type	Constraints
tutorID	Integer	Primary Key, required
userID	Foreign Key Users(userID)	Required
subjectID	Foreign Key Subjects(subjectID)	Required
bio	String	Required
qualifications	String	Required
availability	String	Optional, possibly fully booked

STUDENTS

Field	Data Type	Constraints
-------	-----------	-------------

studentID	Integer	Primary Key, required
userID	Foreign Key Users(userID)	Required
grade	String	Required
school	String	Required
progress_notes	String	Optional, possibly has not had a lesson yet
status	Enum: 'active', 'archived', 'recently inactive', 'blocked'	Default = "active"

LESSONS

Field	Data Type	Constraints
lessonID	Integer	Primary Key, required
studentID	Foreign Key Students(studentID)	Required
subjectID	Foreign Key Subjects(subjectID)	Required
tutorID	Foreign Key Tutors(tutorID)	Required
date	DateTime	Required
status	Enum: 'pending', 'confirmed', 'completed', 'cancelled'	Default = "pending"

FEEDBACK

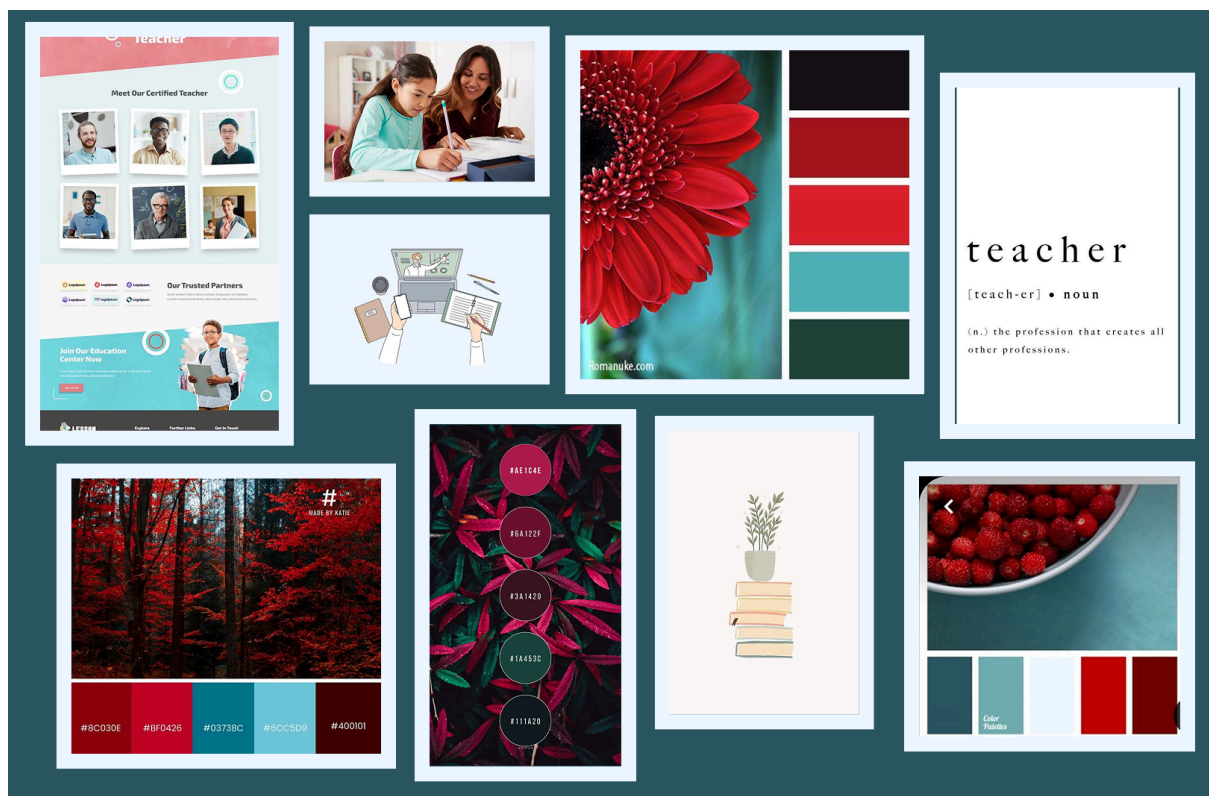
Field	Data Type	Constraints
feedbackID	Integer	Primary Key, required
lessonID	Foreign Key Users(lessonID)	Required
studentID	Foreign Key Users(studentID)	Required
tutorID	Foreign Key Tutors(tutorID)	Required
rating	Integer	Required, between 1 and 5
comment	String	Optional
submitted_at	DateTime	Default = Now

SUBJECTS

Field	Data Type	Constraints
subjectID	Integer	Primary Key, required
name	String	Unique, required
description	String	Optional

F) Wireframes & UI/UX Considerations

Moodboard



Basic wireframes of key screens

Booking

Logo

Subject

Select Date with (Enter)

Pop Up

Pick Time

08:00-09:00

10:00-11:00

Book Lesson

Lesson Request

Logo

Pending Requests

☒ Name Date

Subject Accept / Decline

☒ Name Date

Subject Accept / Decline

Cancelled Requests

☒ Name Date

Subject CANCELLED

Student File View

Logo

Name

School grade

Progress

Status

Tutor Notes

Attended Lessons

23 July - Subject feedback submitted

26 July - Subject feedback submitted

30 July - Subject feedback missing

1 August - Subject feedback reported

Data Management Pages

Received Feedback Lesson Confirmation Page

Logo

Feedback Received for (July)

☒ Name Date Report

☒ Name Date Report

☒ Name Date Report

Student Files

Logo

Students

Sort

Name Name Name

Name Name Name

Post Students

Sort

Name Name Name

Name Name Name

Login

Sign Up / Login

Name

Email

Password

OR

☒ Sign up with Google

Select role

Dashboard (tutor)

Logo

Profile

Username

Email

Bio

Qualifications

Subjects

Pop Up (on selected date)

July, 23

10:00 Math - Carter

16:00 HW - Wainood

Calendar

July

S	M	T	W	T	F	S
29	30	1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	31	1	2
3	4	5	6	7	8	9

Justification for design decisions

The decision to limit extravagant or overly complex features is intentional and rooted in creating a more focused, functional, and user-friendly experience. By prioritizing **core functionality** over flashy elements, the platform ensures that users, especially tutors and students, can navigate the system with ease and confidence.

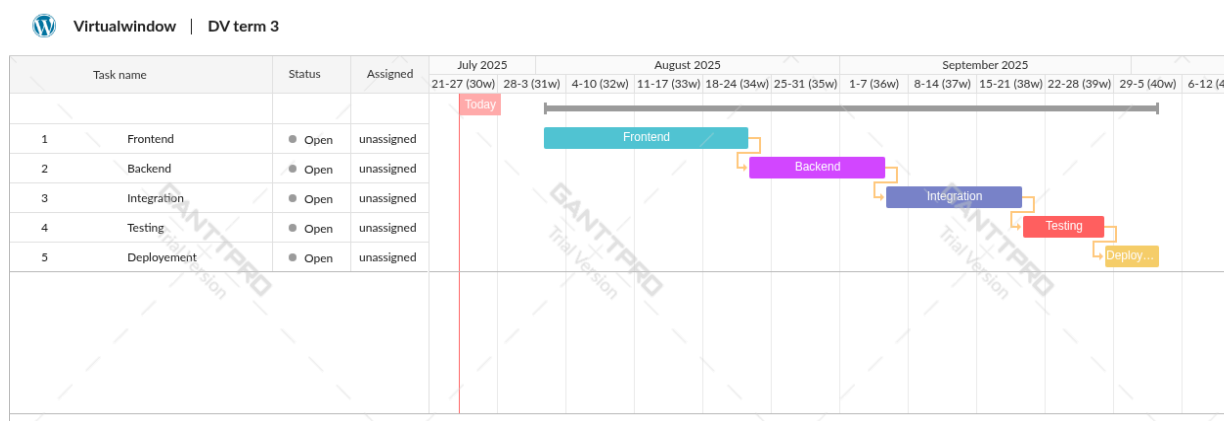
By keeping the design minimal and user-centered, the system achieves a balance between usability and purpose, delivering a better experience to users while staying true to the platform's core objective: making tutoring easier, more accessible, and professionally managed.

Accessibility concerns

All users, even those with visual, motor, or cognitive disabilities, can utilise the platform because it was created with accessibility in mind. Using high contrast colours, providing screen readers with semantic HTML and appropriate labels, guaranteeing full keyboard navigation, and keeping a responsive layout for various devices are all important factors to take into account. The interface uses straightforward language and is clear of clutter to lessen cognitive stress. Features like captions for audio and video content will also be taken into account in future upgrades. To guarantee an inclusive user experience, continuous enhancements will be guided by accessibility testing.

G) Project Timeline & Workflow

High-level Gantt chart



Milestones and Deliverables

Milestone 1: Project Proposal, Planning and System Design

Deliverables:

- Finalised project brief & goals
- Research on existing tutoring platforms
- Final system idea & scope
- Defined users (e.g. tutor, student, admin)
- Proposal document submitted
- High-Level System Architecture Diagram
- Entity Relationship Diagram (ERD)
- Flowcharts or user journey maps
- Accessibility & UX design considerations
- Data types and constraints table

Milestone 2: Frontend Development

Deliverables:

- Login / signup system
- Tutor dashboard (view lessons, feedback, availability)
- Student view (book tutor, browse subjects)
- Forms: submit feedback

Milestone 3: Backend & Database

Deliverables:

- User authentication (email & password / Firebase)
- REST API / Backend logic (Node.js / Firebase functions)
- Database setup with correct constraints
- Data validation and session tracking

Milestone 4: Testing & Evaluation

Deliverables:

- Functionality testing (frontend + backend)
- Usability testing
- Bug fixes

Milestone 5: Final Submission & Presentation

Deliverables:

- Final working app (deployment)
- Final documentation (code, database, design, testing)
- Final presentation s

Estimated Timelines

Backend	(2 weeks)
Frontend	(2 weeks)
Integration	(2 weeks)
Testing	(1 week)
Deployment	(1 week)

Project Management

In order to keep track of progress within the project scope, we will utilise Agile for task management and GitHub for collaboration and repository management.

H) Risks, Challenges & Conclusion**Potential Risks**

1. Server outages - Could prevent bookings and lesson management.
2. Data Loss or Corruption - Loss of data for bookings, progress notes and user accounts.
3. Security Breaches - Possible unauthorised access to user data or accounts.
4. Calendar Conflicts - Double-booking students.
5. Verification Bottleneck - Admins may not be able to verify tutor profiles on time.

6. Report Management Overload - Admins may be bombarded with too many complaints and/or false complaints.
7. Legal & Privacy Concerns - Dealing with personal information creates legal implications.

Mitigation Strategies

1. Server outages - Using a well-known cloud service will help admins monitor the system with fallback notifications..
2. Data Loss or Corruption - Utilise regular database backup.
3. Security Breaches - Use strong authentication (JWT/Firebase), password hashing (bcrypt) and enforce role-based access control.
4. Calendar Conflicts - Add server-side validation and add a way to stop a booking from coming through if that slot is already booked through the backend.
5. Verification Bottleneck - Use an automated email verification to filter, with a manual check later on.
6. Report Management Overload - Ensure there are clear and strict community guidelines, and let users resolve minor problems through feedback and FAQs before submitting a report to the admins.
7. Legal & Privacy Concerns - Follow national data protection regulations like POPIA, including clear terms of service and privacy policy.

Final Thoughts

Tutors currently resort to using tools like WhatsApp, Google Calendar and spreadsheets. TutorAid streamlines most of the usage of these tools by integrating lesson management, tracking availability and progress tracking into one application. It addresses common freelance and private tutor concerns, making it an authentic, practical and usable platform for the day-to-day of a tutor. Utilising frontend-backend integration, calendar management, and user role management provides enough complexity for growth, but is completely realistic for the given timeline. TutorAid helps tutors stay organised, allowing

them to remain professional with fewer booking issues while building their reputations through the feedback from the students. Though there is a set plan for the development of the app, there is still room for expansion, like implementing group season booking, payments and an in-app chat system.